

实验二原生应用开发

一、实验环境

1.1 开发环境

硬件环境

- 开发设备：Windows 10 操作系统
- 测试设备：Android 模拟器 / iOS 模拟器 / H5 浏览器
- 内存要求：8GB RAM 及以上
- 存储空间：20GB 可用空间

软件环境

- 开发工具：HBuilderX 4.0+
- 开发框架：Uniapp + Vue.js 3
- 数据库：IndexedDB（浏览器本地数据库）
- 版本控制：Git
- 调试工具：Chrome DevTools / 微信开发者工具

依赖管理

- 包管理器：npm / yarn
- 主要依赖：
 - Vue.js 3.x
 - Uniapp 框架
 - UI 组件库（自定义）

1.2 技术栈说明

本项目采用 Uniapp 跨平台开发框架，基于 Vue.js 3 构建，实现一次开发、多端运行。

技术选型理由：

1. 跨平台能力：Uniapp 支持编译到 iOS、Android、H5、小程序等多个平台
2. 开发效率：Vue.js 响应式数据绑定，组件化开发，提高开发效率
3. 学习成本低：基于 Web 技术栈，前端开发者容易上手
4. 生态完善：丰富的插件市场和社区支持

二、功能实现

2.1 系统架构设计

系统采用分层架构设计，分为数据层、业务逻辑层和展示层：

- 1. **数据层**：使用 IndexedDB 实现本地数据持久化，存储设备信息、告警记录、用户数据等
- 2. **业务逻辑层**：封装数据库操作、数据验证、状态管理等核心功能
- 3. **展示层**：Vue 组件实现用户界面，响应式更新数据

2.2 核心功能模块

2.2.1 用户认证模块

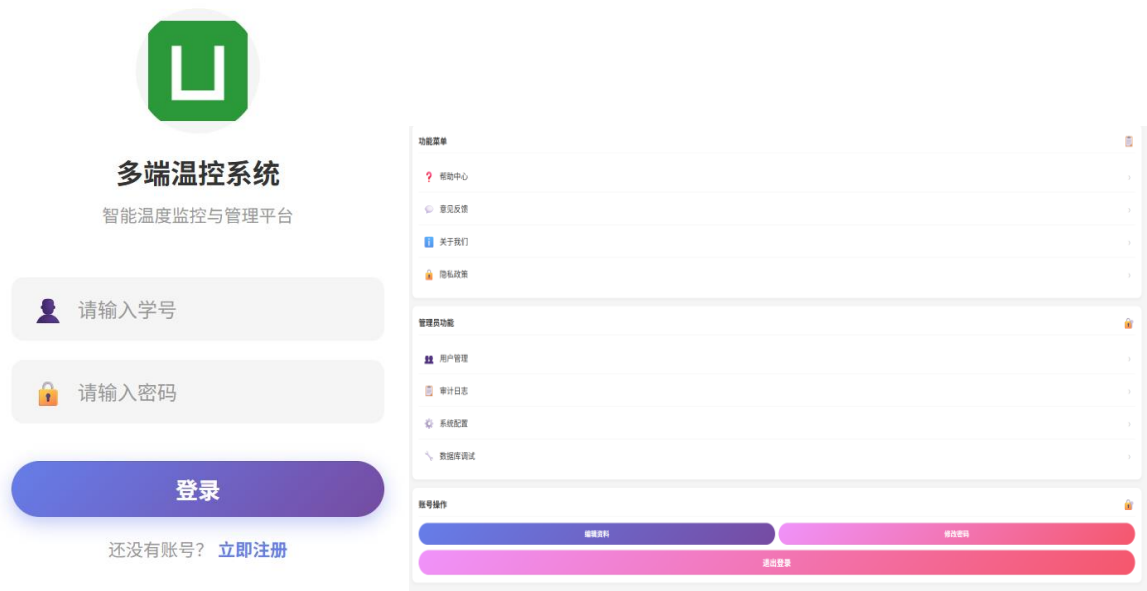
功能描述：实现用户登录、退出登录、权限验证，及个人中心等功能。

实现要点：

- 使用本地存储保存登录状态
- 登录表单验证（用户名、密码非空检查）
- 登录成功后跳转到主页
- 支持退出登录并清除登录状态

技术实现：

- 使用 `uni.setStorageSync()` 保存用户信息
- 使用 `uni.getStorageSync()` 读取登录状态
- 使用 `uni.reLaunch()` 实现页面跳转



图一 登录及个人中心

2.2.2 主页仪表盘模块

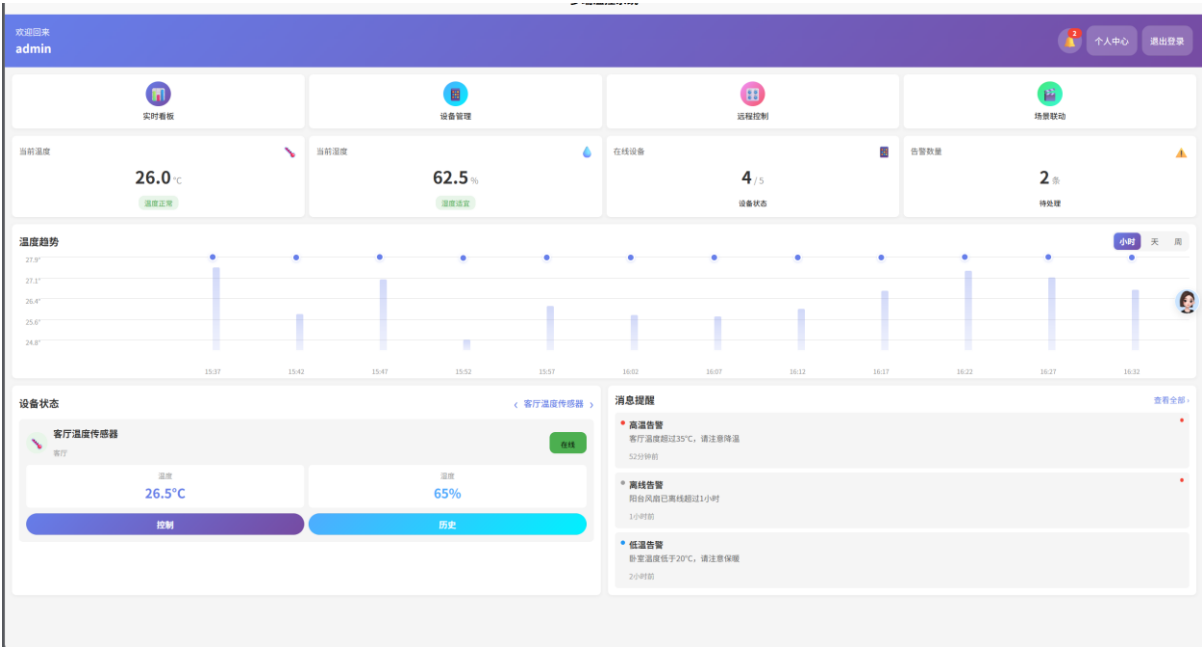
功能描述：展示系统核心数据概览，包括温度、湿度、设备状态、告警信息等。

实现要点：

- 四个概览卡片：当前温度、当前湿度、在线设备、告警数量
- 温度趋势图表：支持小时/天/周三种时间范围切换
- 设备状态展示：支持左右切换查看不同设备
- 消息提醒栏：显示最近 3 条告警消息

技术实现：

- 使用 Vue 计算属性实时计算平均值
- 使用 SVG 绘制温度趋势图（柱形图+折线图组合）
- 使用 CSS Grid 实现响应式布局
- 实现图表坐标轴标签动态生成



图二 主页仪表盘

2.2.3 设备管理模块

功能描述：实现设备的增删改查功能，支持设备状态监控。

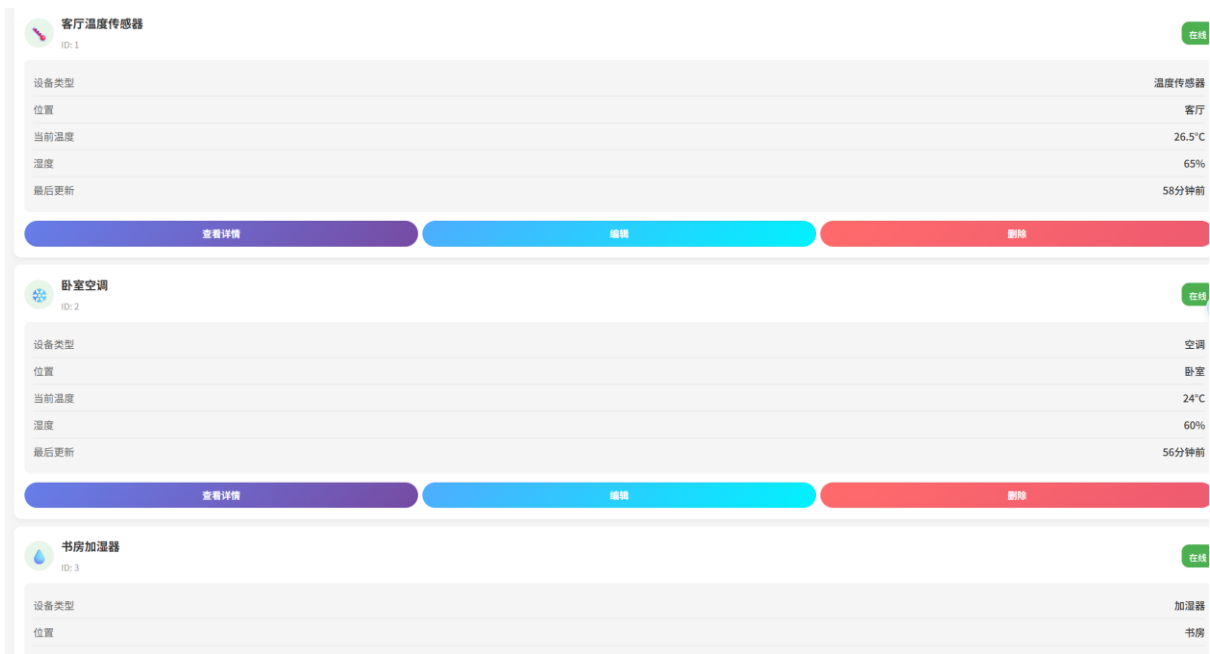
实现要点：

- 设备列表展示：显示设备名称、类型、位置、状态
- 设备添加/编辑：表单验证、数据保存
- 设备删除：确认提示、数据清理
- 设备状态：在线/离线状态标识

技术实现：

- 封装 database.js 提供统一的数据库操作接口

- 使用 `uni.showModal()` 实现确认对话框
- 使用 `uni.showToast()` 显示操作结果提示
- 实现设备图标根据类型动态显示



图三 设备管理

2.2.4 实时看板模块

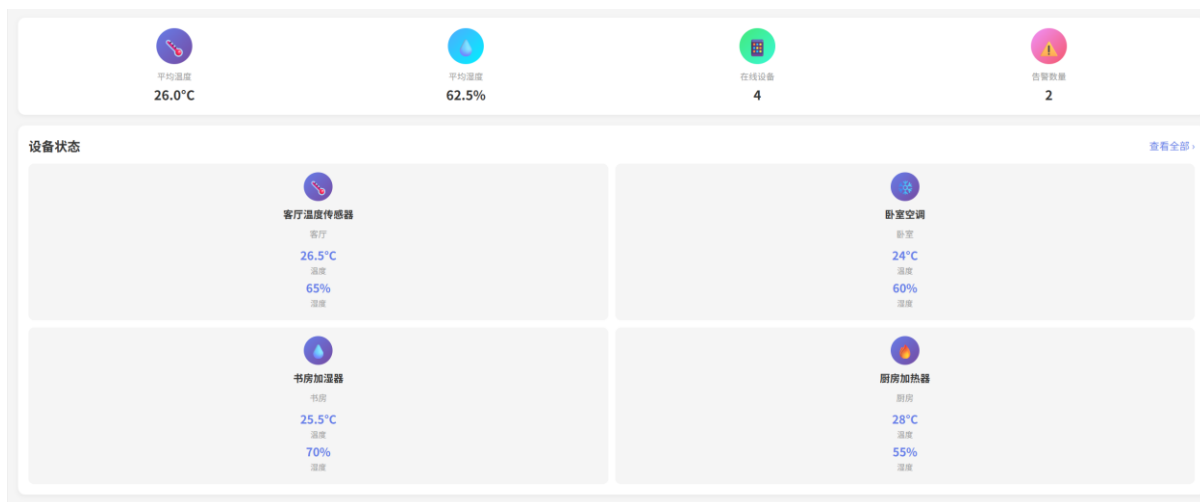
功能描述：实时展示所有设备的温度、湿度数据，支持数据可视化。

实现要点：

- 实时数据更新：定时刷新设备数据
- 温度/湿度曲线图：展示历史数据趋势
- 设备状态卡片：显示每个设备的详细信息
- 数据筛选：按设备、时间范围筛选

技术实现：

- 使用 `setInterval()` 实现定时刷新
- 使用 SVG 绘制数据曲线
- 实现数据筛选和排序功能
- 响应式布局适配不同屏幕



图四 实时看板

2.2.5 远程控制模块

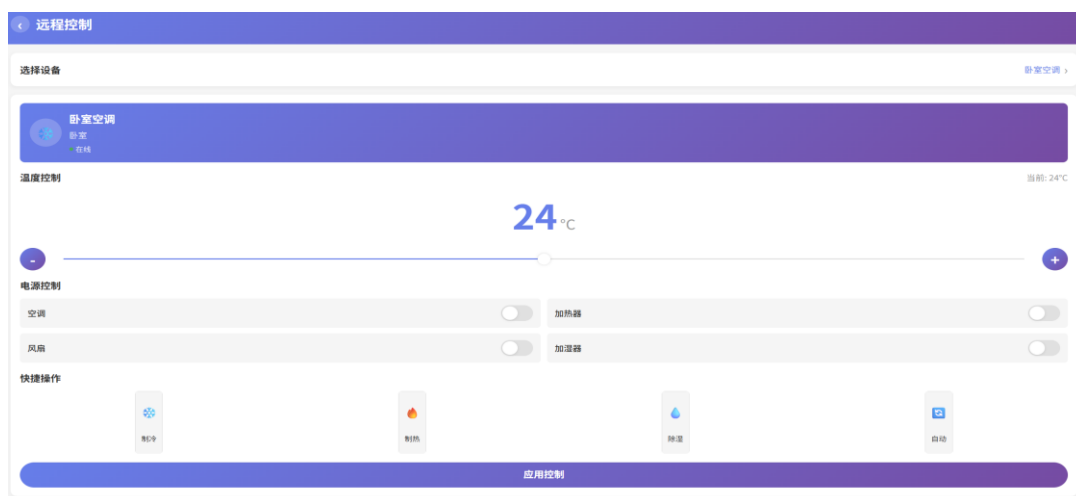
功能描述：实现设备的远程控制功能，包括开关控制、参数调节等。

实现要点：

- 设备控制面板：显示当前设备状态
- 开关控制：切换设备开关状态
- 参数调节：温度、湿度等参数设置
- 控制记录：保存操作历史

技术实现：

- 使用 Vue 双向绑定实现状态同步
- 封装控制操作接口
- 实现控制日志记录
- 提供控制反馈提示



图五 远程控制

2.2.6 场景联动模块

功能描述：实现智能场景配置，支持条件触发和自动执行。

实现要点：

- 场景列表：展示已配置的场景
- 场景创建：设置触发条件和执行动作
- 场景编辑：修改场景配置
- 场景执行：手动触发或自动触发

技术实现：

- 设计场景数据结构（条件+动作）
- 实现条件判断逻辑
- 封装动作执行接口
- 提供场景管理界面



图六 场景联动

2.2.7 告警通知模块

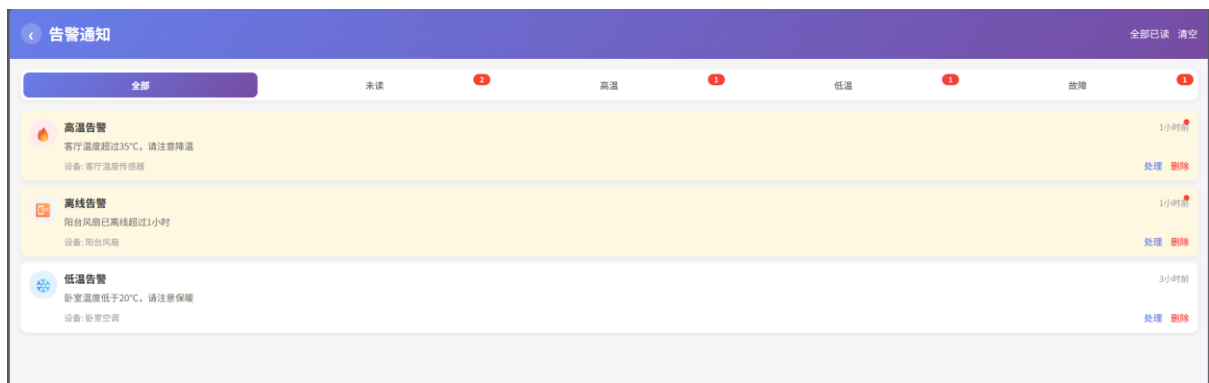
功能描述：接收和展示系统告警信息，支持告警管理。

实现要点：

- 告警列表：展示所有告警记录
- 告警分类：高温、低温、故障、离线等
- 告警状态：未读/已读标识
- 告警操作：标记已读、清空告警

技术实现：

- 实现告警数据结构
- 封装告警生成接口
- 实现告警状态管理
- 提供告警筛选功能



图七 告警通知

三、遇到的问题及解决方案

3.1 数据库初始化问题

问题描述:

首次运行应用时, IndexedDB 数据库未初始化, 导致数据操作失败。

解决方案:

1. 在应用启动时检查数据库是否存在
2. 如果不存在, 执行初始化脚本
3. 创建必要的对象存储 (设备、告警、用户等)
4. 添加示例数据, 方便用户快速体验

具体实现:

- 封装 `initDatabase()` 函数, 检查数据库版本
- 使用 `database.createObjectStore()` 创建存储
- 在 `onLoad()` 生命周期中调用初始化函数

3.2 图表坐标轴显示问题

问题描述:

温度趋势图缺少坐标轴标签, 无法直观看出具体数值和时间。

解决方案:

1. 添加 Y 轴标签: 在左侧显示温度刻度
2. 添加 X 轴标签: 在底部显示时间刻度
3. 调整图表布局: 为坐标轴预留空间
4. 实现标签动态计算: 根据数据范围自动生成刻度

具体实现:

- 使用 `getYAxisLabel()` 函数计算 Y 轴刻度
- 使用 `chartData.time` 作为 X 轴标签

- 调整图表容器高度和 padding
- 使用绝对定位放置坐标轴标签

3.3 页面布局适配问题

问题描述:

在不同屏幕尺寸下，页面布局显示不一致，部分内容被遮挡。

解决方案:

1. 使用 CSS Grid 实现响应式布局
2. 设置合理的间距和内边距
3. 使用百分比和相对单位 (rpx)

具体实现:

- 使用 `grid-template-columns: repeat(4, 1fr)` 实现四列布局
- 使用 `gap: 12rpx` 设置间距
- 使用 `rpx` 单位适配不同屏幕

3.4 返回键功能缺失问题

问题描述:

所有页面缺少返回键，用户无法返回上一页，导航体验不佳。

解决方案:

1. 为每个页面添加返回按钮
2. 统一返回按钮样式：圆形、半透明背景
3. 调整页面头部布局，为返回键预留空间

具体实现:

- 使用 flex 布局调整头部元素位置

四、心得体会

4.1 技术选型思考

通过本次实验，我深刻体会到技术选型对项目开发的重要性。选择 Uniapp 作为跨平台开发框架，主要基于以下考虑：

1. **开发效率：**Uniapp 基于 Vue.js，前端开发者可以快速上手，大大提高了开发效率。相比原生开发需要分别学习 Kotlin 和 Swift，Uniapp 的一套代码多端运行的优势非常明显。
2. **跨平台能力：**Uniapp 支持编译到 iOS、Android、H5、小程序等多个平台，真正实现了"一次开发，多处运行"。这对于资源有限的团队来说，可以大幅降低开发和维护成本。
3. **生态完善：**Uniapp 拥有丰富的插件市场和社区支持，遇到问题时可以快速找到解决

方案。同时，HBuilderX 开发工具提供了完善的调试和打包功能，提升了开发体验。

4. 性能表现：虽然跨平台框架在性能上略逊于原生开发，但对于温控系统这类应用来说，性能完全满足需求。Uniapp 的优化机制使得应用在移动端运行流畅。

4.2 开发过程中的收获

4.2.1 数据库设计的重要性

在实现 IndexedDB 数据持久化时，我深刻体会到数据库设计的重要性。良好的数据结构设计可以简化业务逻辑，提高数据查询效率。本项目中，我设计了设备、告警、用户等多个数据表，并建立了合理的关联关系，使得数据操作变得简单高效。

4.2.2 组件化开发的优势

Vue.js 的组件化开发模式让我受益匪浅。通过将页面拆分为多个可复用的组件，不仅提高了代码的可维护性，还减少了重复代码。例如，概览卡片、消息项等组件可以在多个页面中复用，大大提高了开发效率。

4.2.3 响应式布局的实践

在实现页面布局时，我学会了使用 CSS Grid 和 Flexbox 实现响应式布局。通过合理使用百分比和相对单位（rpx），使得应用在不同屏幕尺寸下都能良好显示。这对于移动端应用来说尤为重要，因为用户使用的设备屏幕尺寸各不相同。

4.2.4 数据可视化的实现

在实现温度趋势图时，我学习了使用 SVG 绘制图表。相比使用第三方图表库，SVG 更轻量、更灵活。通过手动计算坐标和绘制元素，我对数据可视化的原理有了更深入的理解。同时，柱形图和折线图的组合展示，使得数据更加直观。

4.3 遇到的困难与解决

在开发过程中，我遇到了不少困难，但通过查阅文档、搜索资料和反复调试，最终都得到了解决。这个过程让我学会了如何独立解决问题，提高了我的调试能力。

特别是图表坐标轴和折线图的实现，让我对 SVG 和 CSS 定位有了更深入的理解。通过不断调整参数和测试效果，最终实现了满意的视觉效果。

五、实验结论

本次实验成功实现了一个功能完善的多端温控系统，包括用户认证、主页仪表盘、设备管理、实时看板、远程控制、场景联动、阈值设置、告警通知、历史数据、管理员功能等核心模块。

系统采用 Uniapp 跨平台开发框架，基于 Vue.js 3 构建，使用 IndexedDB 实现数据持久化。通过合理的系统架构设计和组件化开发，实现了代码的可维护性和可扩展性。

实验过程中，我遇到了数据库初始化、图表显示、布局适配、数据同步等问题，但通过查阅文档、反复调试，最终都得到了解决。这些问题的解决过程，让我对移动应用

开发有了更深入的理解。并且我掌握了 **Uniapp** 和 **Vue.js** 的使用，学会了如何设计良好的系统架构、解决实际问题。同时，我也认识到技术选型、代码规范等重要性。